

Tratamento de Excecoes e Falhas

Simulador Estrategico de Loja | Squad 16

Validacao de dados, seguranca, banco de dados e resiliencia

Versao 1.0 | Abril 2026

Este documento descreve todos os mecanismos de tratamento de erros e excecoes implementados no projeto. Como a aplicacao nao utiliza APIs externas de IA nem servicos de terceiros pagos, o foco esta nos erros de validacao, autenticacao, autorizacao, banco de dados, logica de negocio e resiliencia interna.

Indice

1. Visao Geral da Estrategia de Erros
2. Erros de Validacao (Zod)
3. Erros de Autenticacao e Autorizacao
4. Erros de Banco de Dados (Prisma)
5. Erros de Logica de Negocio
6. Tratamento de Excecoes no Frontend
7. Resiliencia: Transacoes e Rollback
8. Producao vs Desenvolvimento
9. Tabela Consolidada de Erros
10. Ausencia de APIs Externas e Servicos de IA

1. Visao Geral da Estrategia de Erros

O backend adota um padrao de tratamento centralizado de erros. Todos os controllers sao encapsulados por `asyncHandler()`, que captura excecoes de Promises e as encaminha ao middleware global `errorMiddleware` via `next(err)`. Isso elimina `try/catch` repetitivo nos controllers e garante que nenhum erro nao tratado "quebre" o servidor silenciosamente.

Padrao `asyncHandler` (backend/src/utils/asyncHandler.js):

```
// utils/asyncHandler.js
function asyncHandler(fn) {
  return (req, res, next) => {
    Promise.resolve(fn(req, res, next)).catch(next);
  };
}
// Todos os controllers: module.exports = asyncHandler(minhaFuncao)
```

Middleware global (backend/src/middlewares/errorMiddleware.js):

```
// middlewares/errorMiddleware.js
function errorMiddleware(err, req, res, next) {
```

```
const isProd = process.env.NODE_ENV === "production";

if (err instanceof ZodError) // Validacao
  return res.status(400).json({ error: "Dados invalidos", details: err.flatten() });

if (err.code === "P2025") // Prisma: nao encontrado
  return res.status(404).json({ error: "Registro nao encontrado" });

if (err.code === "P2002") // Prisma: unique constraint
  return res.status(409).json({ error: "Registro ja existe" });

console.error("[errorMiddleware]", err);
return res.status(500).json({
  error: "Erro interno do servidor",
  ...(isProd ? {} : { message: err.message, stack: err.stack }),
});
}
```

Fluxo de Propagacao de Erros

Controller lanca execucao -> asyncHandler().catch(next) -> errorMiddleware(err, req, res, next) -> resposta HTTP padronizada

2. Erros de Validacao (Zod)

Toda entrada de dados do usuario e validada com Zod antes de qualquer operacao no banco de dados. A validacao ocorre nos controllers via safeParse() para controle manual, e no frontend via zodResolver() integrado ao React Hook Form.

2.1 Validacao no Backend (safeParse)

```
// Exemplo: authController.js
const loginSchema = z.object({
  email: z.string().email("Email invalido"),
  password: z.string().min(6, "Senha deve ter no minimo 6 caracteres"),
});

const parsed = loginSchema.safeParse(req.body);
if (!parsed.success) {
  return res.status(400).json({ errors: parsed.error.flatten().fieldErrors });
}

// Resposta de erro retornada:
// { "errors": { "email": ["Email invalido"] } }
```

2.2 Schemas de Validacao por Recurso

Endpoint	Campos Validados
POST /auth/login	email (email), password (min 6)
POST/PUT /users	name (min 1), email, password (min 6), role (enum)
POST/PUT /products	name (min 1), purchasePrice (positivo)
POST/PUT /squads	name (min 1)
PUT /inventory/:productId	quantity (int >= 0)
POST /inventory/restock	items[].productId (uuid), items[].quantity (int >= 0)
POST /rounds	number (int pos), startDate, endDate (coerce date)

POST /rounds/:id/config

fixedExpenses (>=0), variableExpenses (>=0), items[].salePrice/salesVolume

2.3 Validacao no Frontend (React Hook Form + Zod)

O frontend valida os formularios antes de disparar qualquer requisicao HTTP, proporcionando feedback imediato ao usuario sem custo de rede.

```
// RoundConfigPage.jsx - schema de validacao
const schema = z.object({
  fixedExpenses: z.coerce.number().min(0, "Nao pode ser negativo"),
  variableExpenses: z.coerce.number().min(0, "Nao pode ser negativo"),
  items: z.array(z.object({
    productId: z.string(),
    salePrice: z.coerce.number().positive("Deve ser positivo"),
    salesVolume: z.coerce.number().int().positive("Deve ser inteiro positivo"),
  })).min(1),
});

// Campos exibem mensagem de erro inline ao lado do input
// Cor do campo muda para vermelho (border-red-400 bg-red-50)
```

2.4 Alertas Visuais de Negocio no Frontend

Alem da validacao de formato, o frontend exibe alertas de negocio em tempo real:

- Preco de venda abaixo do custo: campo fica laranja com mensagem 'Abaixo do custo'
- Volume acima do estoque disponivel: campo fica laranja com mensagem 'Acima do estoque'
- Esses alertas sao visuais; o backend tambem limita effectiveVolume ao estoque real

3. Erros de Autenticacao e Autorizacao

3.1 authMiddleware — Validacao do JWT

```
// middlewares/authMiddleware.js
const authHeader = req.headers.authorization;
if (!authHeader || !authHeader.startsWith("Bearer ")) {
  return res.status(401).json({ message: "Token nao fornecido" });
}
try {
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  req.user = { id: decoded.userId, role: decoded.role, squadId: decoded.squadId };
  return next();
} catch {
  return res.status(401).json({ message: "Token invalido ou expirado" });
}
```

Cenario	HTTP	Mensagem
Header Authorization ausente ou mal-formado	401	"Token nao fornecido"
Token expirado (exp claim)	401	"Token invalido ou expirado"
Assinatura JWT invalida	401	"Token invalido ou expirado"
Role do usuario nao autorizado	403	"Acesso negado"
Player tentando acessar loja de outro squad	403	"Acesso negado: loja nao pertence ao seu squad"

3.2 Interceptor Axios — Tratamento 401 no Frontend

```
// frontend/src/services/api.js
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      useAuthStore.getState().logout(); // limpa token e estado
      window.location.href = "/login"; // redireciona
    }
    return Promise.reject(error); // propaga para o catch local
  }
);
```

Nota de Seguranca

O logout automatico por 401 garante que tokens expirados nao fiquem armazenados no localStorage. O estado Zustand e limpo antes do redirecionamento.

4. Erros de Banco de Dados (Prisma ORM)

O errorHandler intercepta codigos de erro do Prisma Client antes de gerar uma resposta 500 generica. Os erros mais comuns sao tratados explicitamente.

Codigo Prisma	Descricao	HTTP	Resposta
P2002	Unique constraint violation	409	Registro ja existe
P2025	Record not found (update/delete)	404	Registro nao encontrado
Outros	Erros de schema, conexao, etc.	500	Erro interno do servidor

Banco de Dados Local vs Producao

Em desenvolvimento local usa SQLite (prisma/schema.local/prisma + local.db). Em producao usa PostgreSQL v14+. O Prisma abstrai as diferencas, mas erros de conexao com Postgres (ECONNREFUSED, timeout) chegam como erro 500 no middleware.

5. Erros de Logica de Negocio

Alem dos erros tecnicos, os controllers validam regras de negocio e retornam codigos HTTP semanticos com mensagens claras.

HTTP	Controller	Gatilho	Mensagem
400	submitConfig	Usuario sem squadId vinculado	Usuario nao pertence a um squad
400	submitConfig	Squad sem loja cadastrada	Seu squad nao possui uma loja cadastrada
400	submitConfig	ProductIds duplicados no array items	Existem produtos duplicados nos items
400	submitConfig	ProductId nao encontrado no BD	Produtos nao encontrados
404	submitConfig	Rodada nao existe	Rodada nao encontrada
409	submitConfig	Rodada com status != OPEN	Nao e possivel submeter: rodada esta CLOSED/PROCESSING
409	submitConfig	Config ja submetida para esta rodada	Voce ja submeteu uma configuracao
409	createRound	Ja existe rodada OPEN/PROCESSING	Encerre a rodada atual antes de criar outra
409	closeRound	Rodada com status != OPEN	Rodada nao pode ser encerrada (status atual)

404	getResults	Rodada nao CLOSED	Resultados disponiveis apenas apos encerramento
403	getInventory	Player acessando loja de outro squad	Acesso negado: loja nao pertence ao seu squad
409	deleteUser	Usuario lider de squad	Transfira a lideranca antes de remover
409	createUser	Email ja cadastrado	Email ja esta em uso

6. Tratamento de Excecoes no Frontend

6.1 Padroes de Tratamento em Paginas

As paginas utilizam tres estados para lidar com carregamento e falhas: loading (exibe Skeleton), loadError (exibe ErrorMessage + botao Retry), e dados carregados com sucesso.

```
// Padrao aplicado em: RoundConfigPage, StoresDashboard,
// AdminDashboard, ProductsManagementPage, etc.

const [loading, setLoading] = useState(true);
const [loadError, setLoadError] = useState("");

async function load() {
  setLoadError("");
  try {
    const data = await Promise.all([api1(), api2()]);
    // atualiza estado
  } catch {
    setLoadError("Nao foi possivel carregar os dados.");
  } finally {
    setLoading(false);
  }
}

if (loading) return <Skeleton />;
if (loadError) return <ErrorMessage message={loadError} onRetry={load} />;
```

6.2 Toast de Notificacao para Erros de Mutacao

```
// Para acoes (submit, delete, restock, etc.)
try {
  await roundService.submitConfig(roundId, data);
  toast.success("Estrategia enviada com sucesso!");
} catch (err) {
  // Extrai mensagem do backend ou usa fallback
  toast.error(
    err.response?.data?.message ??
    err.response?.data?.error ??
    "Erro ao enviar configuracao"
  );
}
```

6.3 Sessao Expirada / Token Invalido

- Axios intercepta 401 automaticamente em qualquer requisicao
- Chama useAuthStore.getState().logout() — limpa localStorage

- Redireciona para /login com window.location.href
- O usuario ve a tela de login sem mensagem de erro tecnica

6.4 Rota Nao Encontrada

```
// App.jsx - rota catch-all
<Route path="*" element={<Navigate to="/login" replace />} />

// Acesso a rota sem permissao -> PrivateRoute redireciona para /unauthorized
// UnauthorizedPage exibe mensagem amigavel
```

7. Resiliencia: Transacoes Prisma e Rollback

O processamento de rodadas utiliza Prisma Transactions (\$transaction) para garantir atomicidade: ou todos os dados de uma loja sao persistidos (FinancialResult + desconto de Inventory), ou nenhum.

```
// simulationService.js - processamento por loja
for (const config of configs) {
  try {
    await prisma.$transaction(async (tx) => {
      // 1. Calcula DRE
      const dre = calcularDRE(roundConfig, items, inventory);
      // 2. Persiste FinancialResult
      await tx.financialResult.create({ data: { ...dre } });
      // 3. Desconta estoque de cada produto
      for (const b of dre.itemBreakdown) {
        await tx.inventory.update({ where: {...}, data: { quantity: { decrement: b.effectiveVolume } } });
      }
    }); // commit automatico ao final
  } catch (err) {
    // Rollback automatico pelo Prisma
    console.error(`Erro ao processar loja ${config.storeId}:`, err.message);
    // Continua para a proxima loja (nao aborta o processamento completo)
  }
}
```

Comportamento em Caso de Falha

Se o processamento de uma loja falhar (ex: constraint violada, erro de rede com BD), o Prisma reverte automaticamente a transacao daquela loja. As demais lojas continuam sendo processadas. O erro e logado no console do servidor mas nao interrompe a rodada.

8. Comportamento: Producao vs Desenvolvimento

```
// errorMiddleware.js
const isProd = process.env.NODE_ENV === "production";
return res.status(500).json({
  error: "Erro interno do servidor",
  ...(isProd ? {} : { message: err.message, stack: err.stack }),
});
```

Aspecto	Desenvolvimento	Producao
---------	-----------------	----------

Mensagem de erro 500	message + stack trace completo	Apenas 'Erro interno do servidor'
Erros do Prisma (P2025/P2002)	Tratados igualmente	Tratados igualmente
Banco de dados	SQLite (arquivo local.db)	PostgreSQL 14+
JWT_SECRET	Variavel .env.local	Variavel de ambiente segura
Stack trace no response	Visivel (debug)	Ocultado

9. Tabela Consolidada de Todos os Erros

HTTP	Onde	Gatilho	Resposta
400	authController	Email/senha invalidos no formato	{ errors: { email/password: [...] } }
400	UserController	Campos obrigatorios ausentes	{ errors: { fieldName: [...] } }
400	inventoryCtrl	quantity nao e inteiro >= 0	{ errors: { quantity: [...] } }
400	roundController	number/startDate/endDate invalidos	{ errors: { field: [...] } }
400	simulationCtrl	fixedExpenses negativo ou items vazio	{ errors: { field: [...] } }
400	simulationCtrl	Usuario sem squad	{ message: 'Nao pertence a um squad' }
400	simulationCtrl	Squad sem loja	{ message: 'Squad nao possui loja' }
400	simulationCtrl	ProductIds duplicados	{ message: 'Produtos duplicados' }
401	authMiddleware	Header Authorization ausente	{ message: 'Token nao fornecido' }
401	authMiddleware	Token expirado ou invalido	{ message: 'Token invalido ou expirado' }
401	authController	Senha incorreta	{ message: 'Credenciais invalidas' }
403	roleMiddleware	Role insuficiente	{ message: 'Acesso negado' }
403	inventoryCtrl	Player acessando loja alheia	{ message: 'Loja nao pertence ao squad' }
404	roundController	Rodada nao encontrada	{ message: 'Rodada nao encontrada' }
404	inventoryCtrl	Produto nao no estoque	{ message: 'Produto nao encontrado no estoque' }
404	simulationCtrl	Resultado nao disponivel	{ message: 'Resultados apos encerramento' }
404	errorMiddleware	Prisma P2025	{ error: 'Registro nao encontrado' }
409	UserController	Email duplicado	{ message: 'Email ja esta em uso' }
409	UserController	Remover lider de squad	{ message: 'Transfira a lideranca' }
409	roundController	Ja existe rodada ativa	{ message: 'Encerre a rodada atual' }
409	roundController	Fechar rodada nao OPEN	{ message: 'Rodada nao pode ser encerrada' }
409	simulationCtrl	Config ja submetida	{ message: 'Ja submeteu configuracao' }
409	simulationCtrl	Rodada nao OPEN	{ message: 'Nao e possivel submeter' }
409	errorMiddleware	Prisma P2002	{ error: 'Registro ja existe' }
500	errorMiddleware	Excecao nao capturada	{ error: 'Erro interno do servidor' }

10. Ausencia de APIs Externas e Servicos de IA

Sem Integracao com APIs Externas ou Servicos de IA

O projeto NAO utiliza nenhuma API de terceiros (pagamentos, e-mail, SMS, mapas) nem servicos de inteligencia artificial (OpenAI, Anthropic, Gemini, etc.). Todo o processamento e local: calculo de DRE, ranking e feedbacks sao algoritmos puramente deterministas em JavaScript/Node.js sem chamadas externas.

Portanto, as categorias de falha listadas abaixo NAO se aplicam a esta versao do projeto:

- Falhas de conexao com API externa (timeout, ECONNREFUSED, rate limit)
- Respostas inesperadas de API de IA (hallucinations, incomplete responses)
- Indisponibilidade de servico de IA (5xx do provedor)
- Latencia de chamadas a APIs de terceiros

- Limites de requisicao (API quota exceeded)
- Erros de autenticacao com servicos externos (API keys expiradas)
- Webhook failures ou retentativas de entrega

Recomendacao para Versoes Futuras

Se integracao com IA for adicionada (ex: sugestao automatica de preco, analise de desempenho por LLM), recomenda-se: circuit breaker para timeout, fallback gracioso (exibir mensagem 'servico indisponivel' ao inves de erro 500), retry com backoff exponencial, e log centralizado de chamadas externas.